

Rotation and Orientation

DongJoon Kim

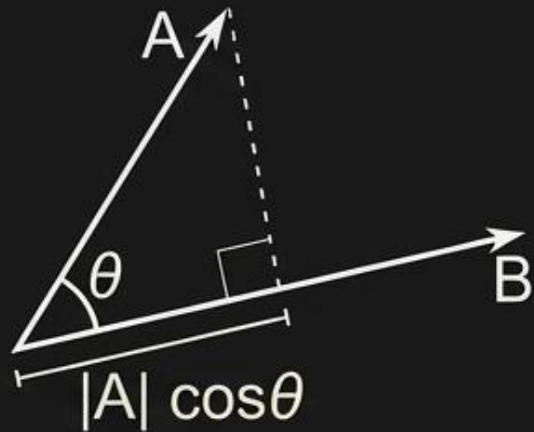
| Phone : +82-2-940-8461

| Email : dongjoonkim@kw.ac.kr

Remind: Axis Convention

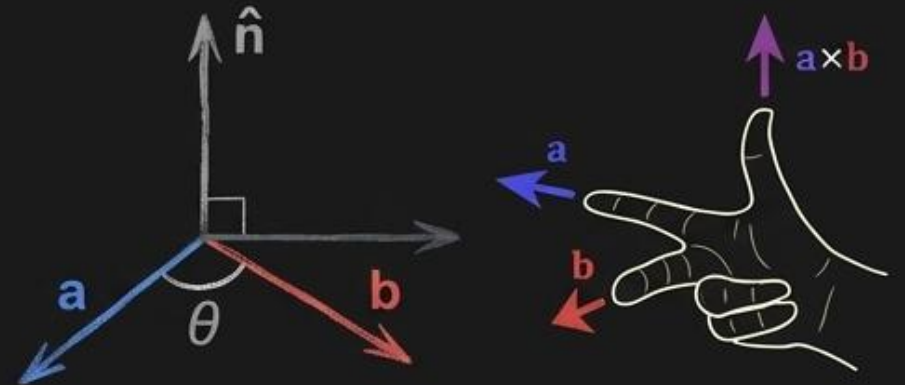
Remind: Vector Algebra

Dot Product ($\cdot$)



- 결과: 스칼라 (Scalar) 반환
- 기하학적 의미: 한 벡터를 다른 벡터에 투영 (Projection)
- 핵심 수식: $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}|\cos(\theta)$

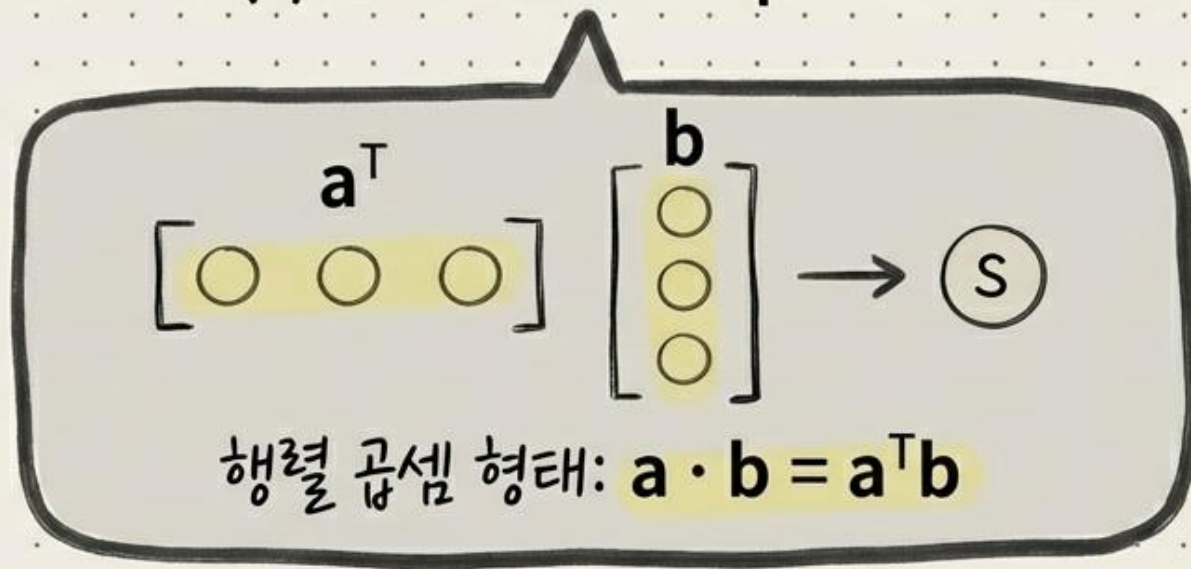
Cross Product ($\times$)



- 결과: 벡터 (Vector) 반환
- 기하학적 의미: 두 벡터가 만드는 평면에 직교하는 법선 벡터 (Normal vector)
- 핵심 방향: 좌표계 방향에 따름 (RHS or LHS)



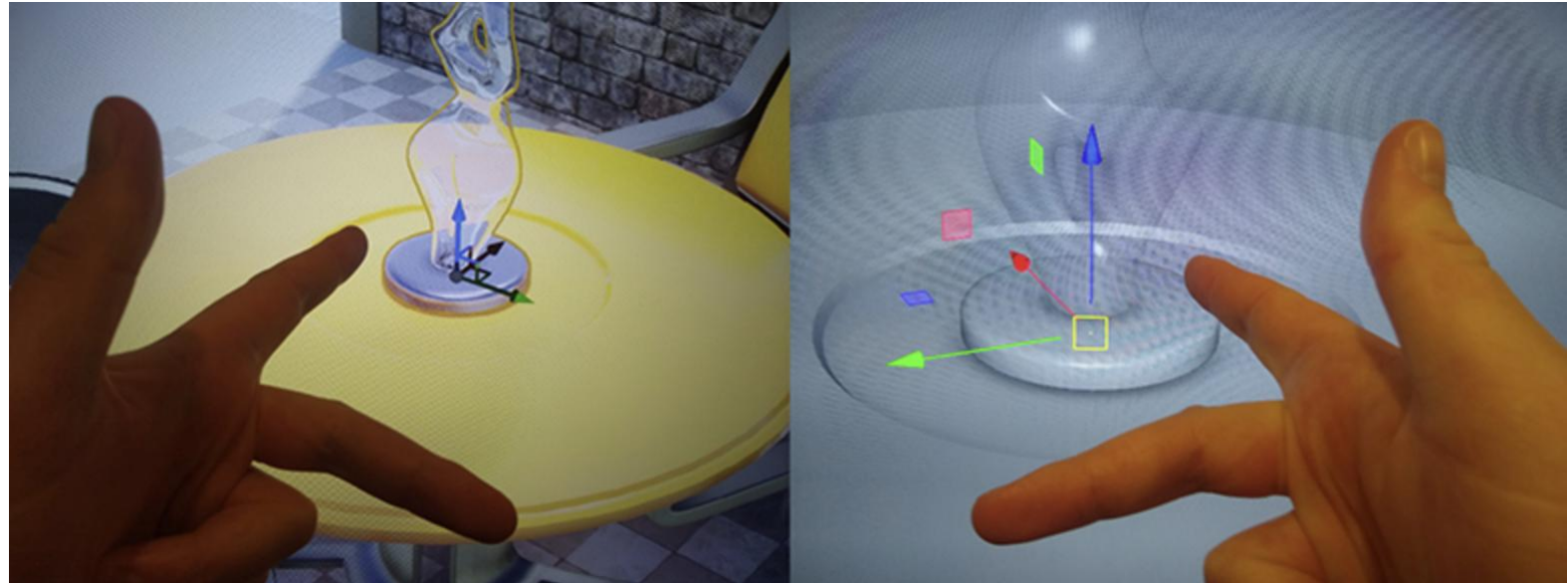
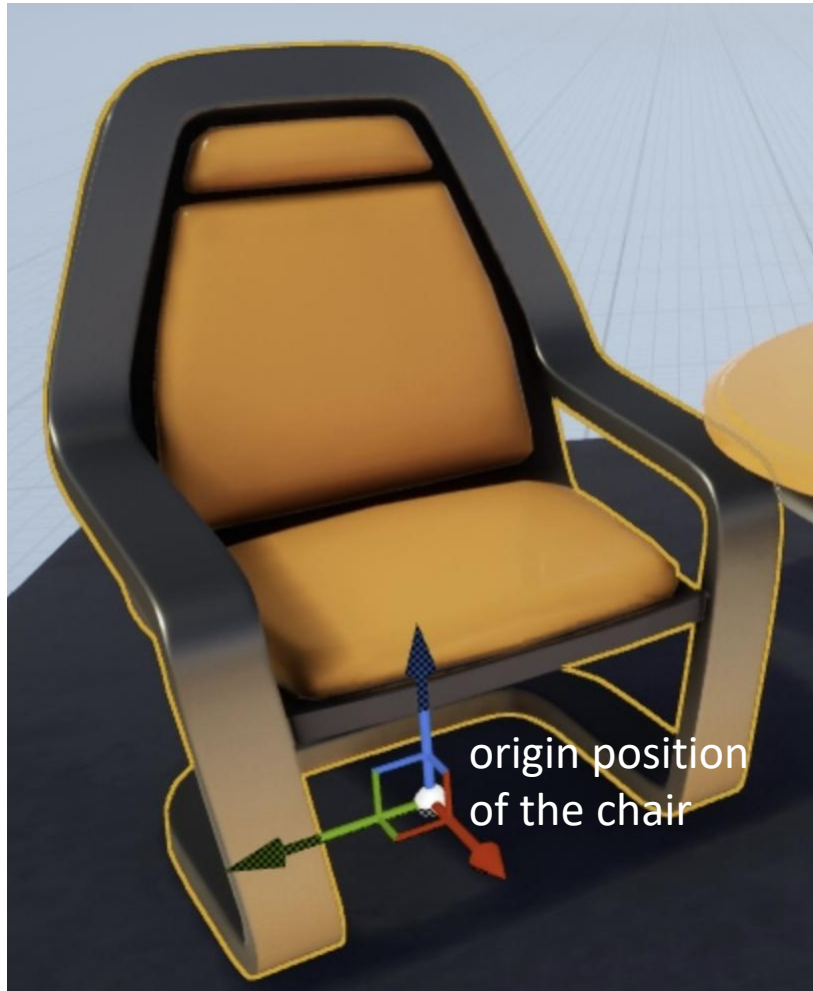
벡터 기호 사이의 ($\cdot$)는 명확히 Dot product를 의미합니다.



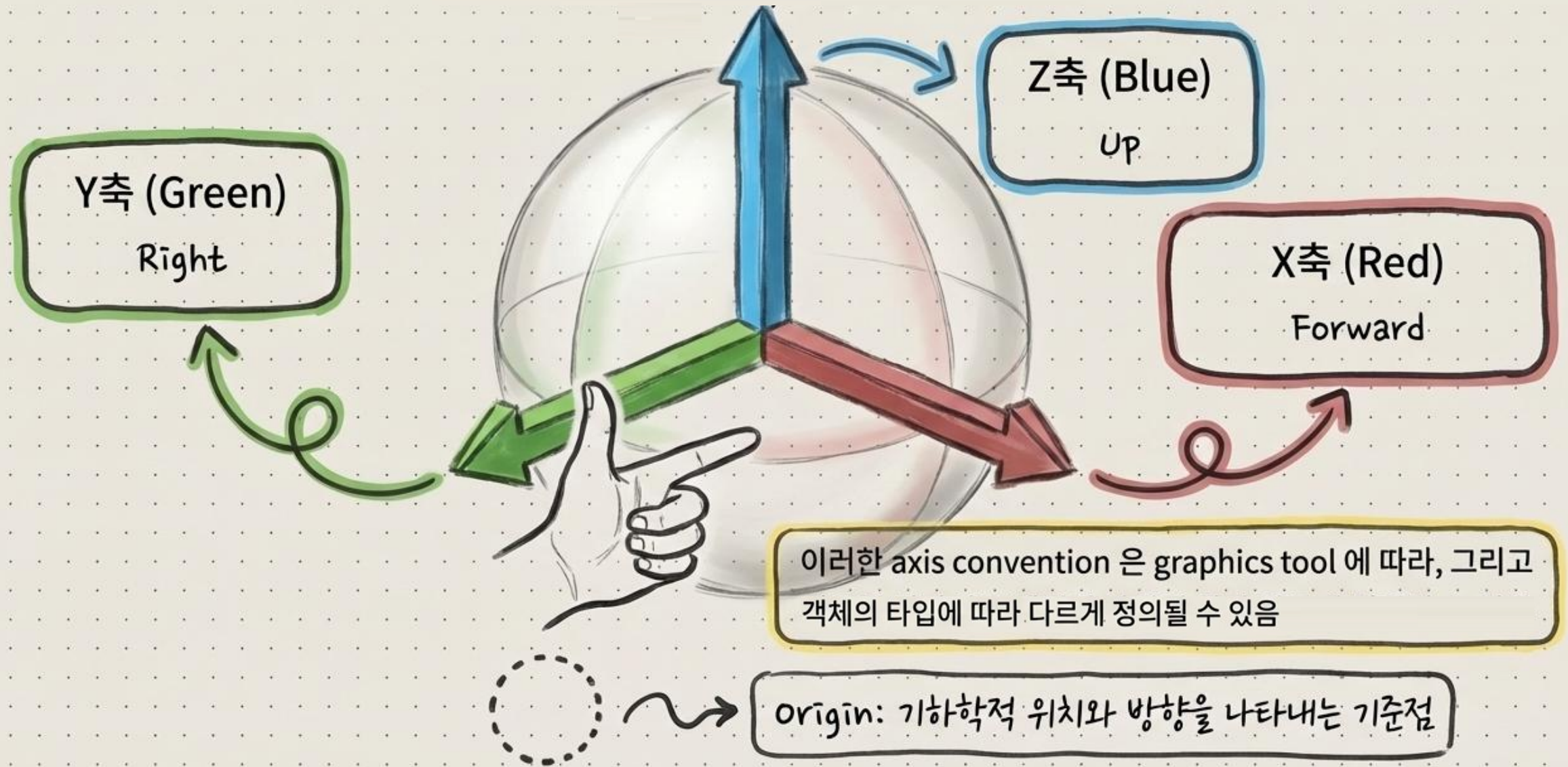
참고: 간혹 $a \cdot b = \langle a, b \rangle$ 형태로 표기하여 혼선을 줄이기도 함.

Axis Convention

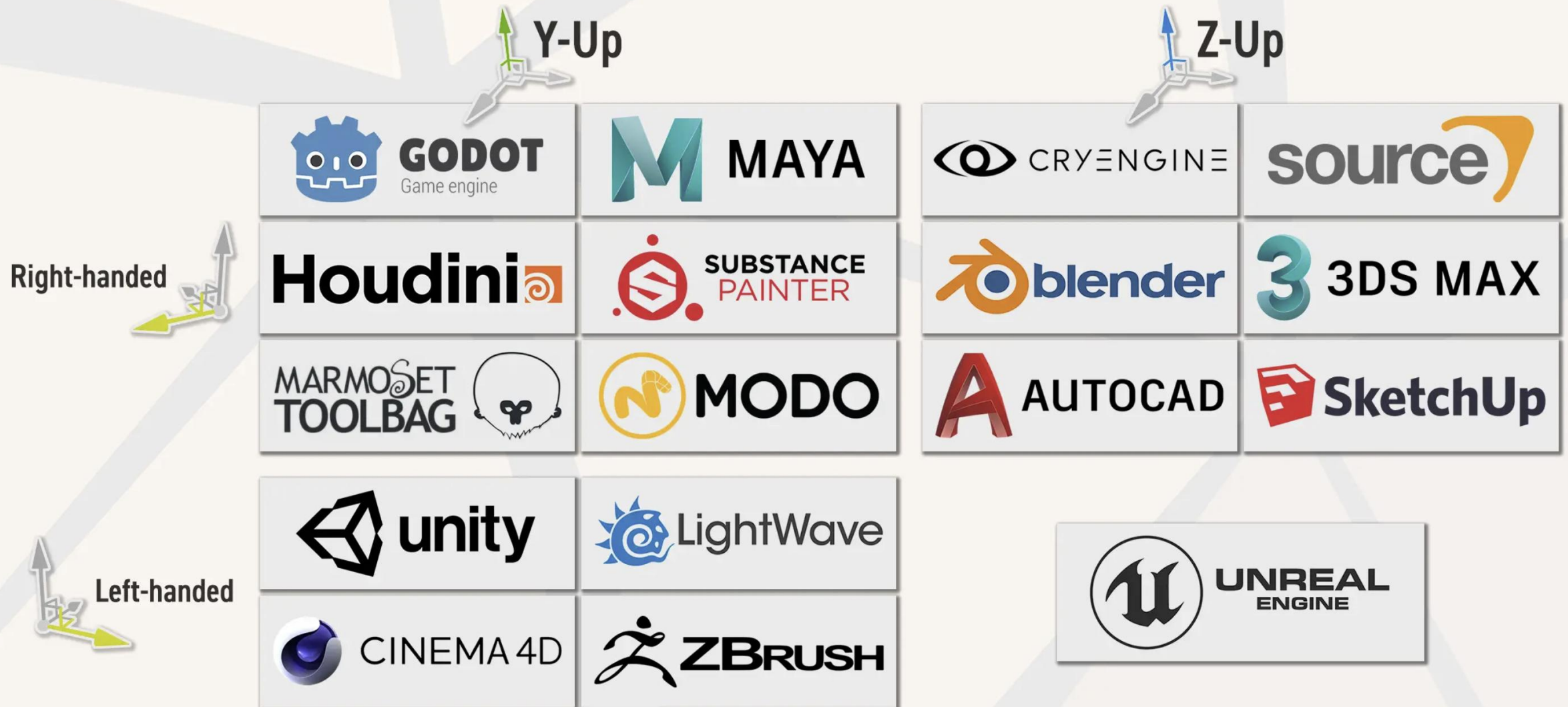
- Axis 시스템 (RHS vs LHS), 각 축이 나타내는 의미는 Graphics Tool 또는 객체 타입에 따라 다를 수 있음



언리얼엔진에서의 Axis Convention

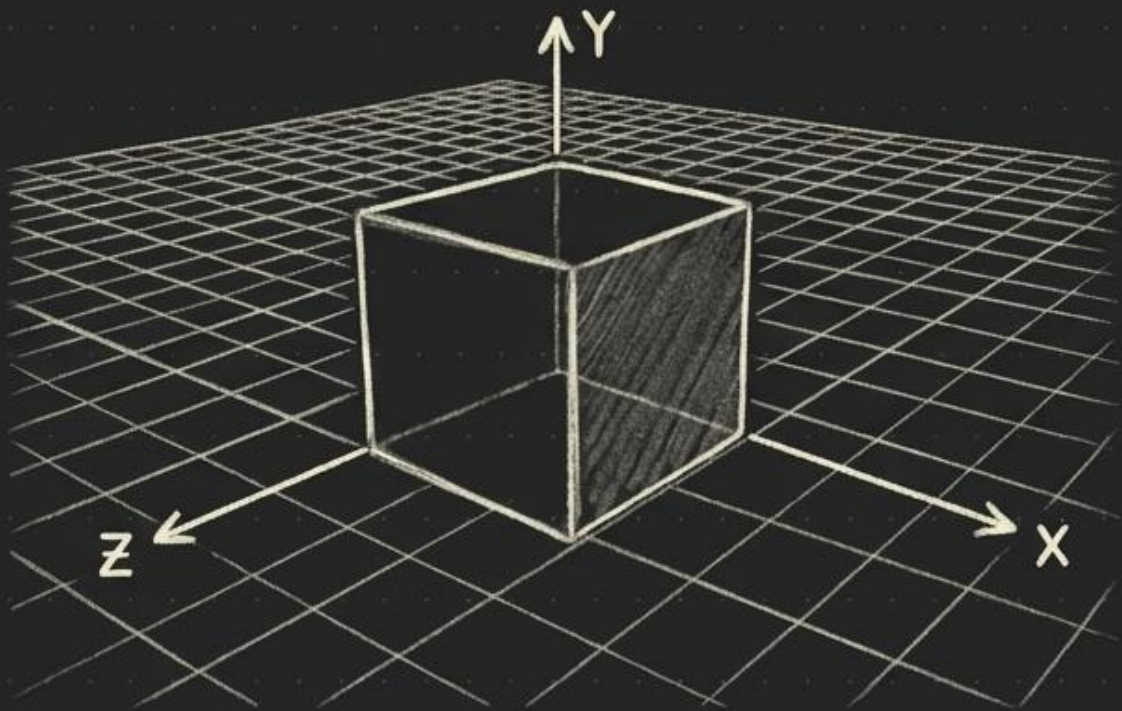


Takeaway: 엔진마다 좌표계 표준이 다르므로, 데이터 импорт/익스포트 시 축 변환(Axis Conversion)이 필수적!



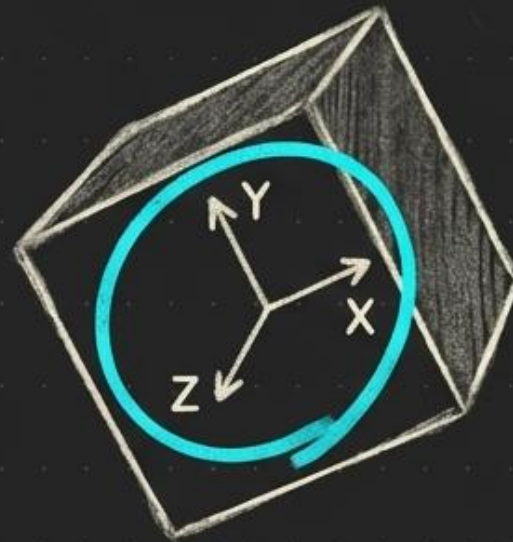
World (Universal) Space and Local (Relative) Space

World Space (절대 공간)



- Fixed coordinate system.
- 환경 전체를 지배하는 변하지 않는 절대적 기준.

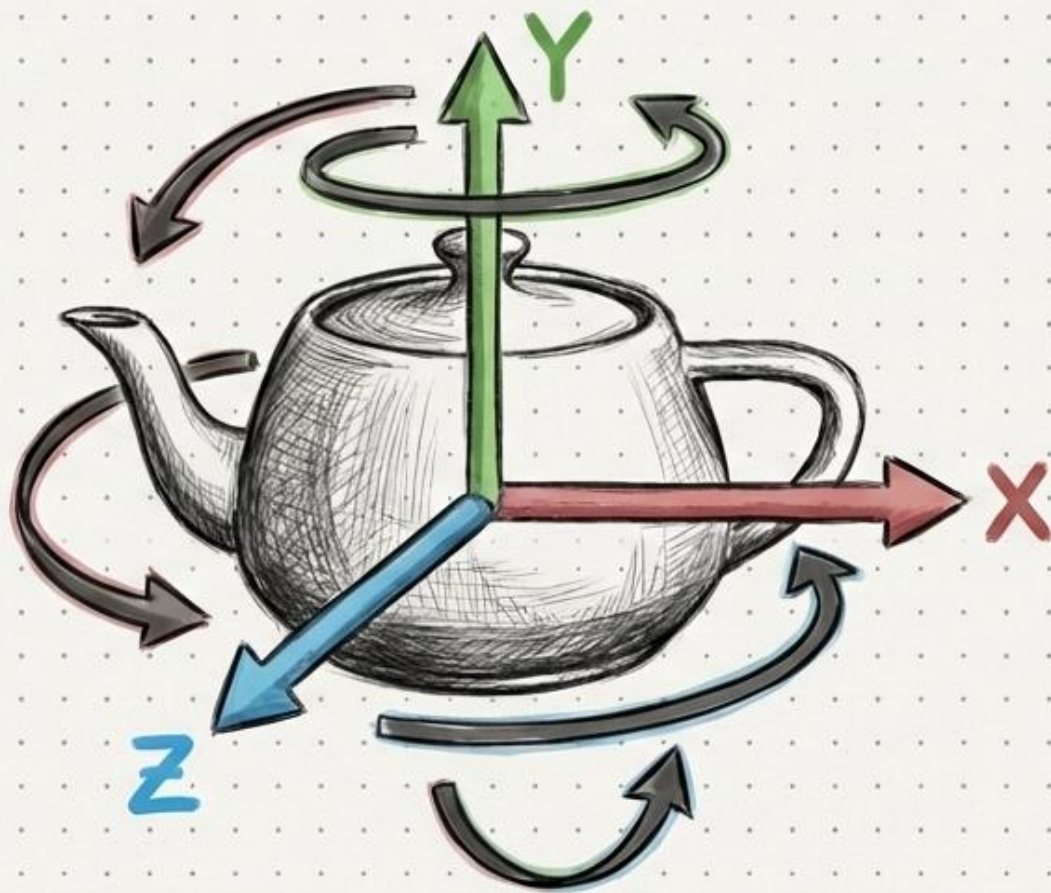
Local Space (상대 공간)



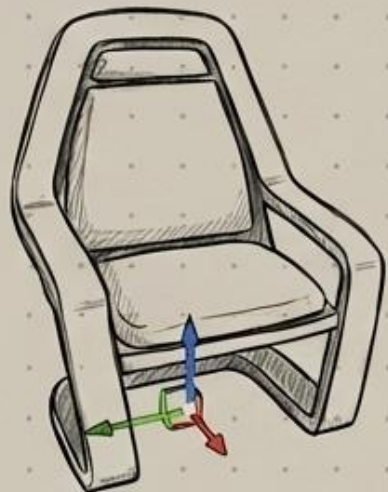
- Rotated coordinate system.
- 개별 오브젝트에 종속되어 오브젝트와 함께 회전하는 기준 축.

3D Rotation and Orientation

수학적 기초부터 Quaternion의 이해까지



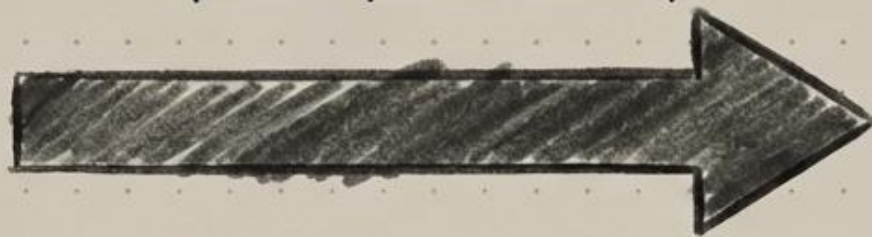
Orientation A
(State/Pose)



명사 : 현재의 기준 프레임(상태)

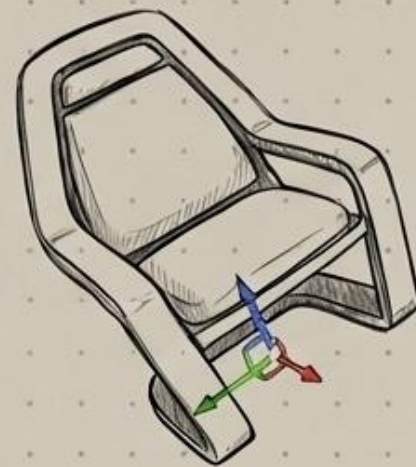


Rotation
(Action/Movement)



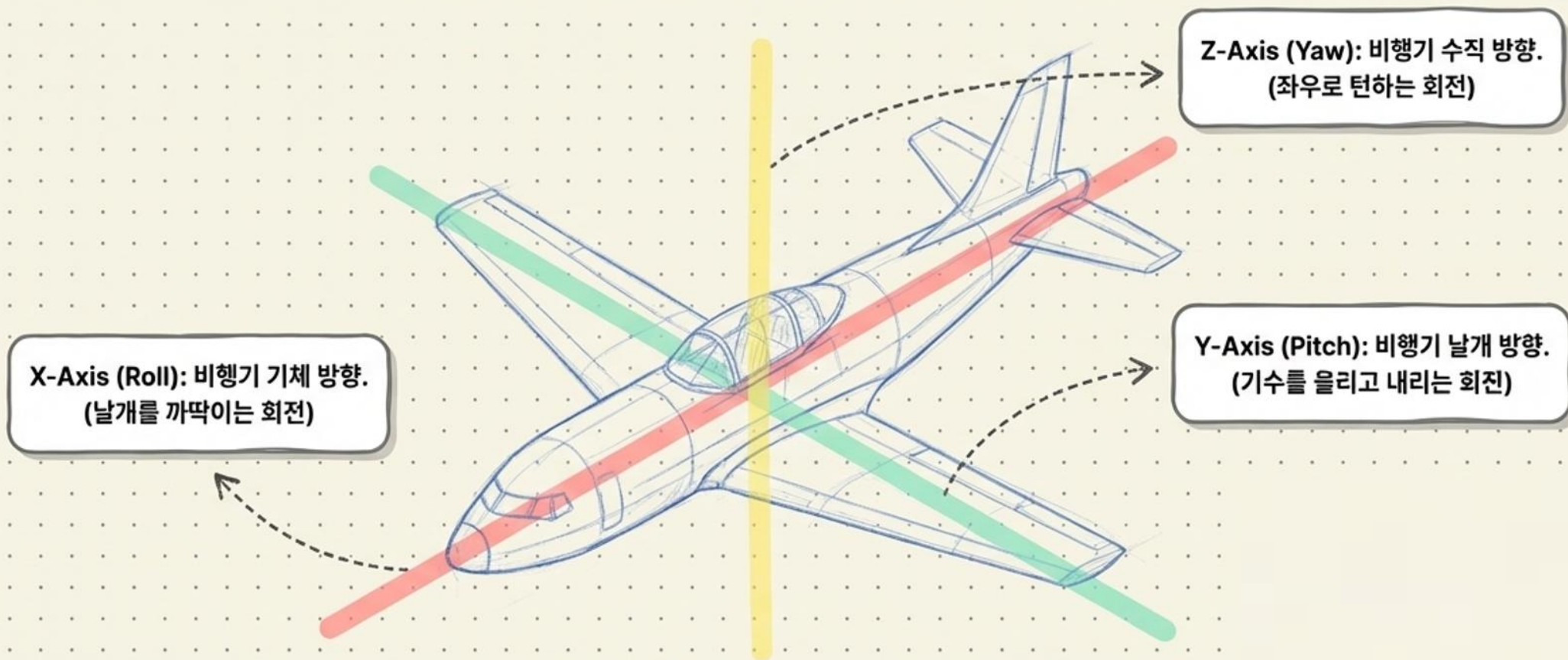
동사 : 상태를 변화시키는 행동

Orientation B
(State/Pose)



Insight : 회전(Rotation)이 없다면,
두 좌표계는 같은 방향(Orientation)을 가진다.

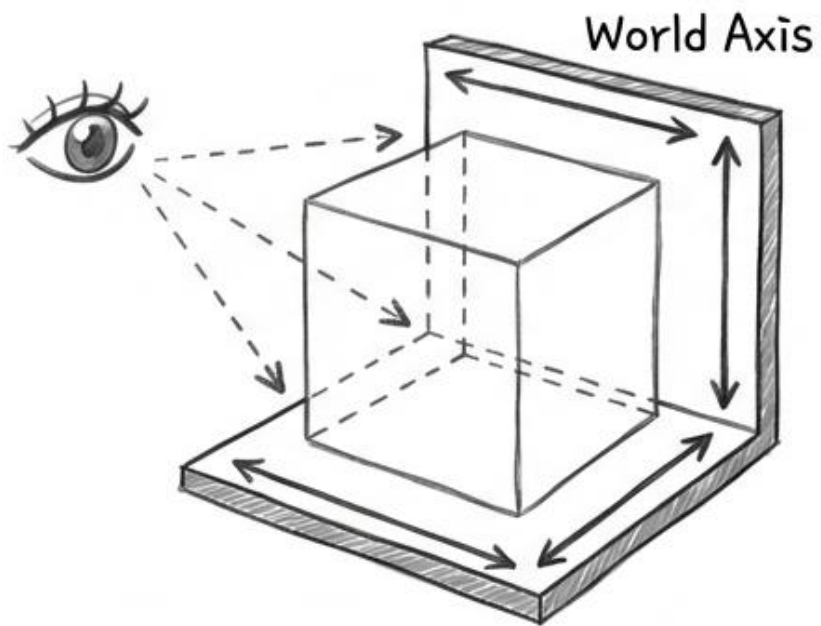
The Intuitive Approach: Euler Angle



Warning: 12가지의 회전 순서 조합(X-Y-Z 등)이 존재하며, 순서에 따라 결과가 완전히 다름.

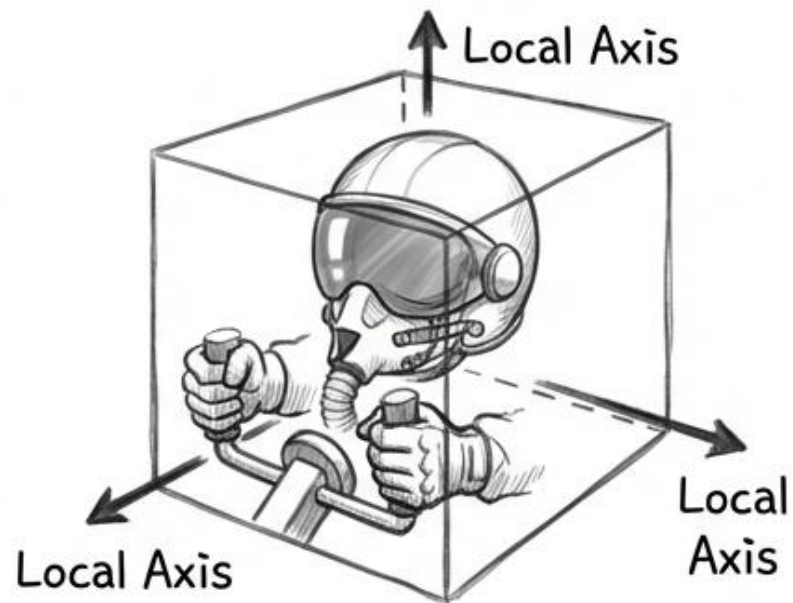
기준 좌표계에 따른 변환 이해

Extrinsic



물체 밖에서 관찰하며, 세상에 고정된 절대적인 축(World Axis)을 기준으로 변환합니다.

Intrinsic

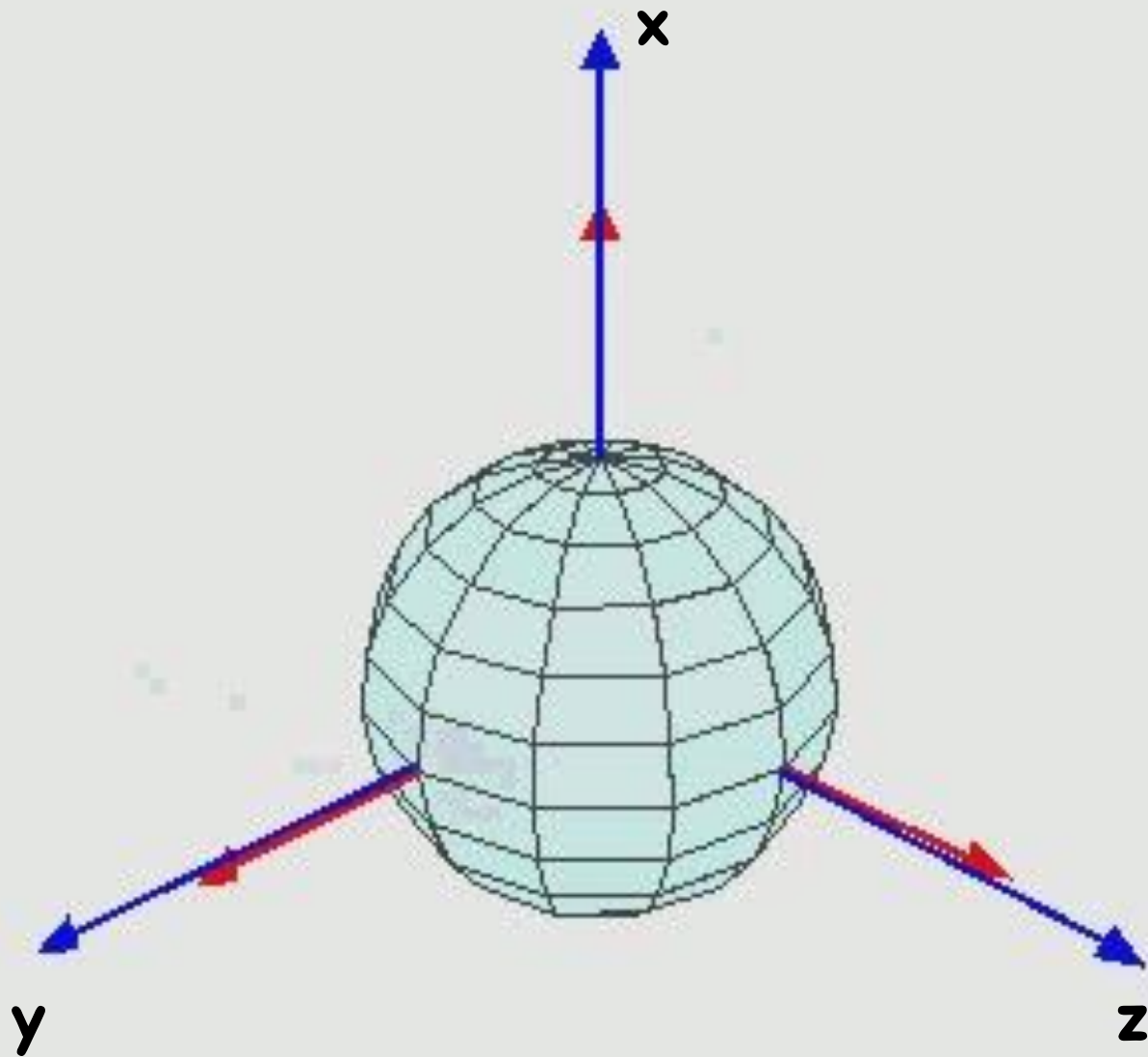


물체 안에 탑승하여, 내 몸과 함께 움직이는 로컬 축(Local Axis)을 기준으로 변환합니다.

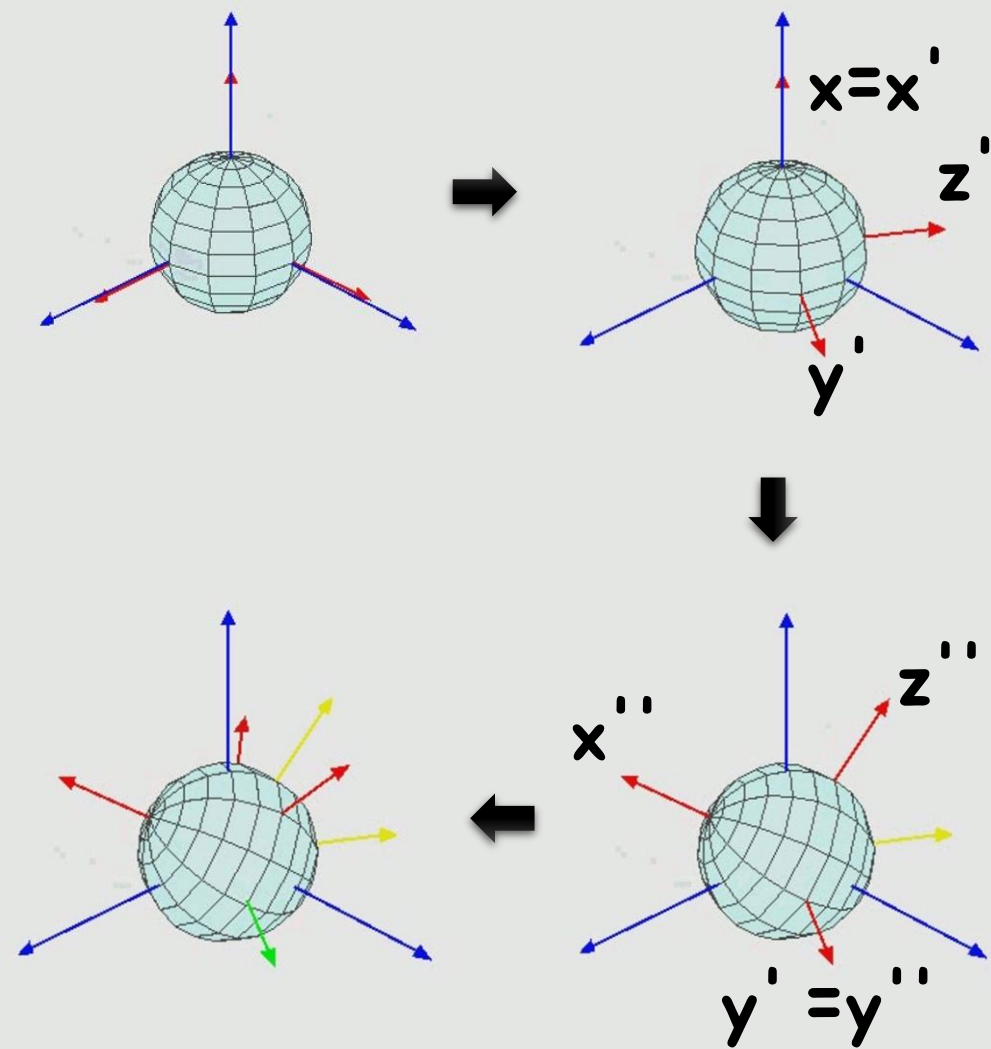
회전 순서를 완전히 뒤집어도 최종 자세는 같아집니다



왜 안에서 순서대로 돌린 결과가
밖에서 역순으로 돌린 결과와 일치할까?



Intrinsic XYZ = Extrinsic ???



행렬을 어느 쪽에 곱하느냐가 회전의 기준을 결정합니다

$R2 \times R1$ (왼쪽에 곱하기)

처음에 정해둔 세상의 축을
기준으로 다시 회전
→ Extrinsic (고정축 기준)

회전 행렬 R

$R1 \times R2$ (오른쪽에 곱하기)

$R1$ 에 의해 변한 좌표계 안에서
다시 회전
→ Intrinsic (현재축 기준)

실전 3D 그래픽스를 위한 회전 변환 행렬 전개식

ZYX 순서 (일반적인 항공/그래픽스 표준)

$$R = R_z(\text{Yaw}) \times R_y(\text{Pitch}) \times R_x(\text{Roll})$$

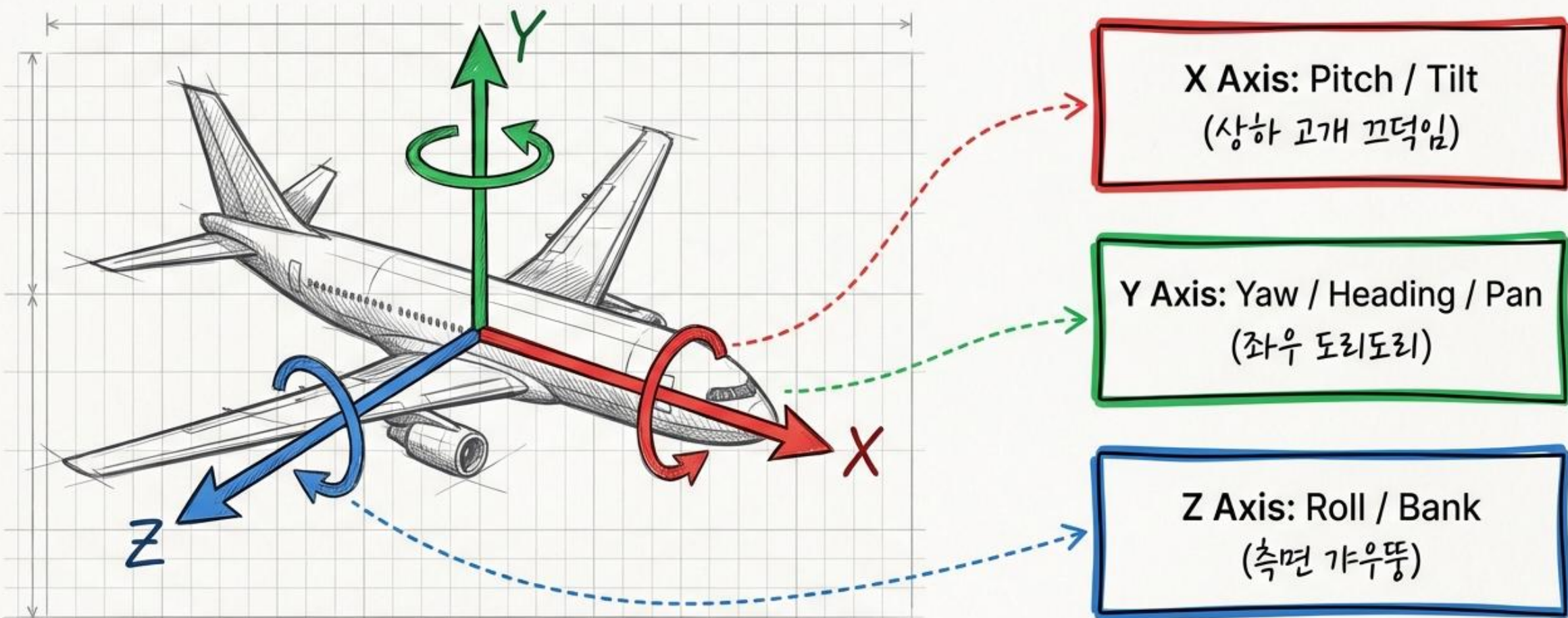

(주석) Extrinsic 기준으로 볼 때, 세상의 축을 바탕으로
X(Roll) → Y(Pitch) → Z(Yaw) 순서로 적용됨을 의미합니다.

XYZ 순서 (대안적 스펙)

$$R = R_x(\text{Roll}) \times R_y(\text{Pitch}) \times R_z(\text{Yaw})$$

Euler Angle and Gimbal Lock Problem

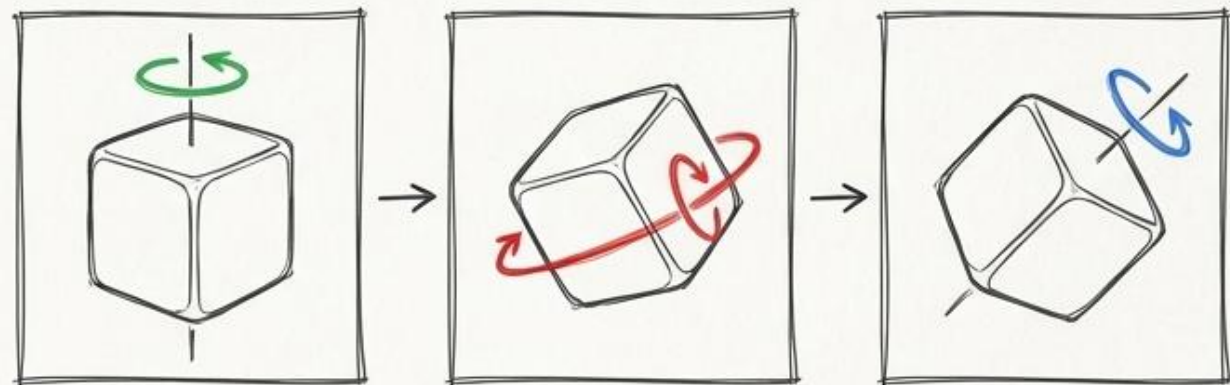
3D Rotation Basics: 3 Degrees of Freedom



3D 소프트웨어는 이 3가지 축의 회전값을 조합하여 공간상의 모든 방향을 표현함.

The Secret of Euler: 순차적 계산

Euler 시스템은 3개의 축이 동시에 회전하는 것이 아닙니다.
설정된 **Rotation Order**에 따라 한 번에 하나씩, 순차적으로 계산됩니다.



이 순차적 특성이 모든 문제(Gimbal Lock)를 일으키는 근본적 원인입니다.

Hierarchy: 회전의 종속성



Parent (최상위 - Y): Y가 회전하면 하위 축(X, Z)을 모두 데리고 돕니다.



Middle (중간 - X): X가 회전하면 상위(Y)는 가만히 있고, 하위(Z)만 데리고 돕니다.



Child (최하위 - Z): Z가 회전하면 오직 자신만 돌며, 다른 축에 영향을 주지 않습니다.

Canonical Rotations: Euler Angles

A typical Euler angle representation

- Roll : rotation about x-axis (forward/backward)
- Pitch : rotation about y-axis (left/right)
- Yaw : rotation about z-axis (up/down)

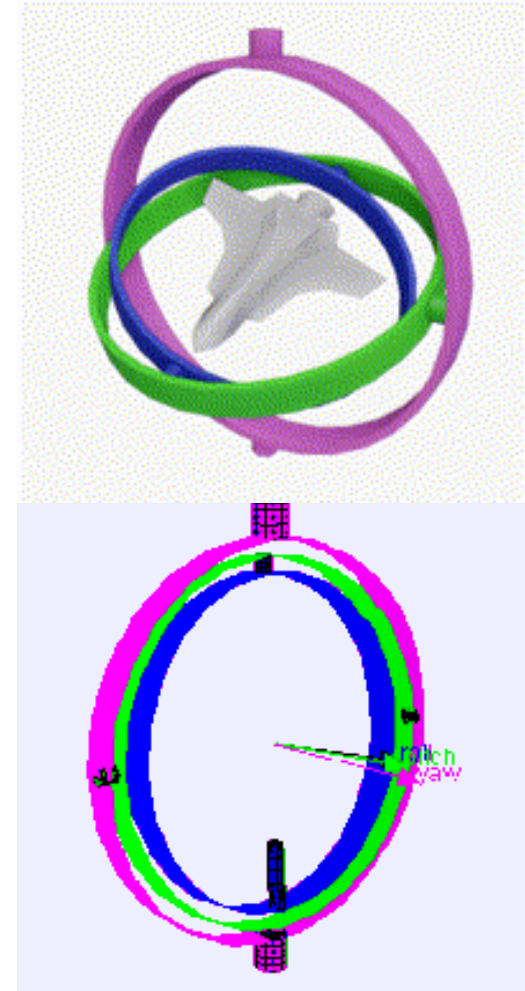
Gimbal Lock

- We can represent any rotation by combining such axis rotations
- **Final rotation depends on the order of** such rotation sequence
- Sometimes, 3-DOF is lost (gimbal lock)
- This makes limitations in the interpolation of orientation
- Gimbal is a good example showing this limitation (loss of DOF)

Worth reading: <https://handhp1.tistory.com/3>
https://youtu.be/XCh-yEkHPMo?si=gQyJ0n4KEDYM9i_2

Gimbal lock problem

➔ Lost of DOF



[wikipedia. Gimbal lock]

i

q

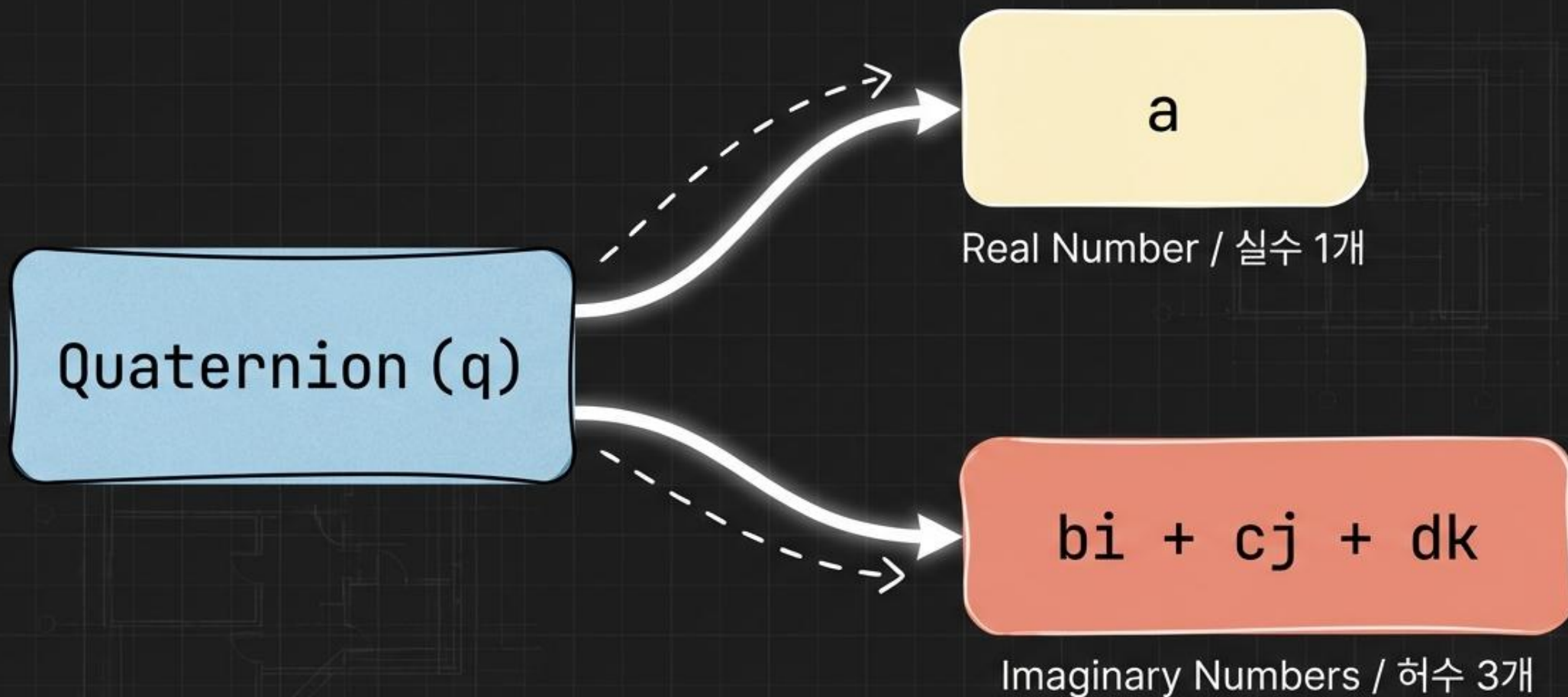
j

Understanding Quaternion

3D Rotation과 Interpolation을 위한 핵심 개념 정리

θ

k



3D 공간의 Rotation(회전)을 표현하는 강력한 수학 도구.

Unit Quaternion (단위 쿼터니언)

모든 Quaternion이 Rotation은 아님!



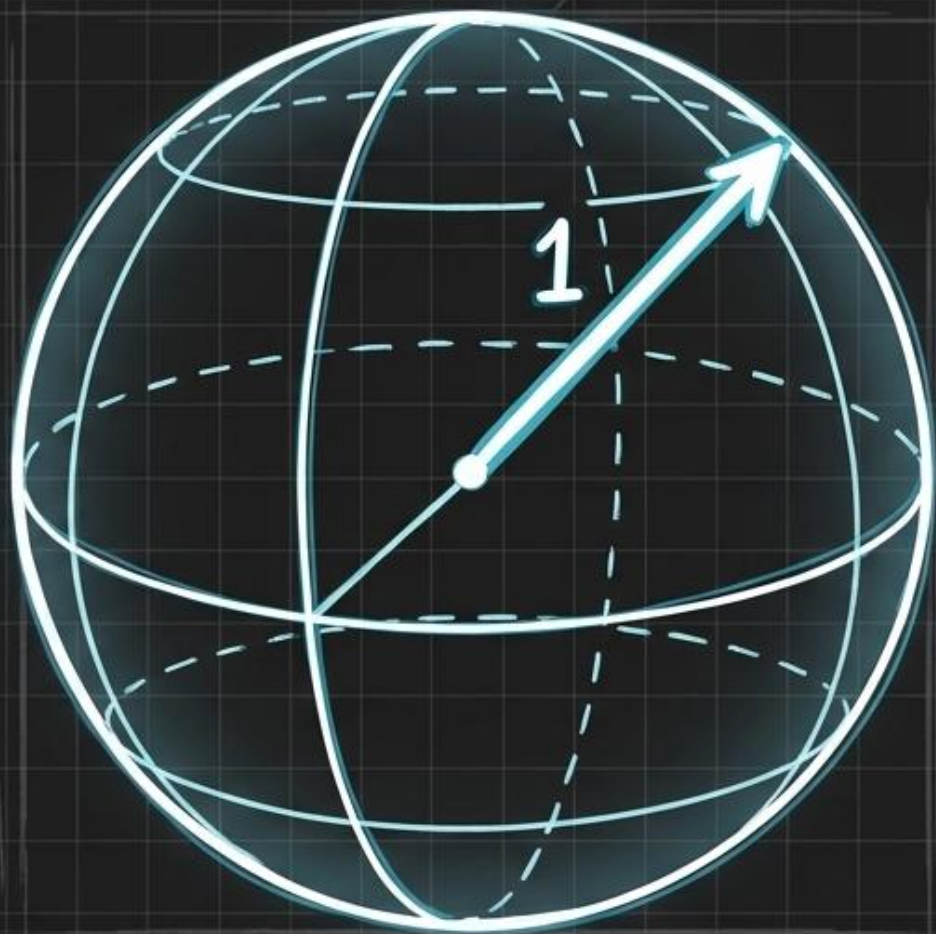
길이가 1인 경우에만 Rotation을 의미.



$$|\mathbf{q}|^2 = a^2 + b^2 + c^2 + d^2 = 1$$



if $\mathbf{q} = 1$, then $\mathbf{u} = 1$



Rotation Vector
(회전 축)

$$q = \cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right)u$$

Rotation Angle
(회전 각도)



역방향 회전 (Inverse) -> 부호만 반대로!
 $q^{-1} = \cos(\theta/2) - \sin(\theta/2)u$

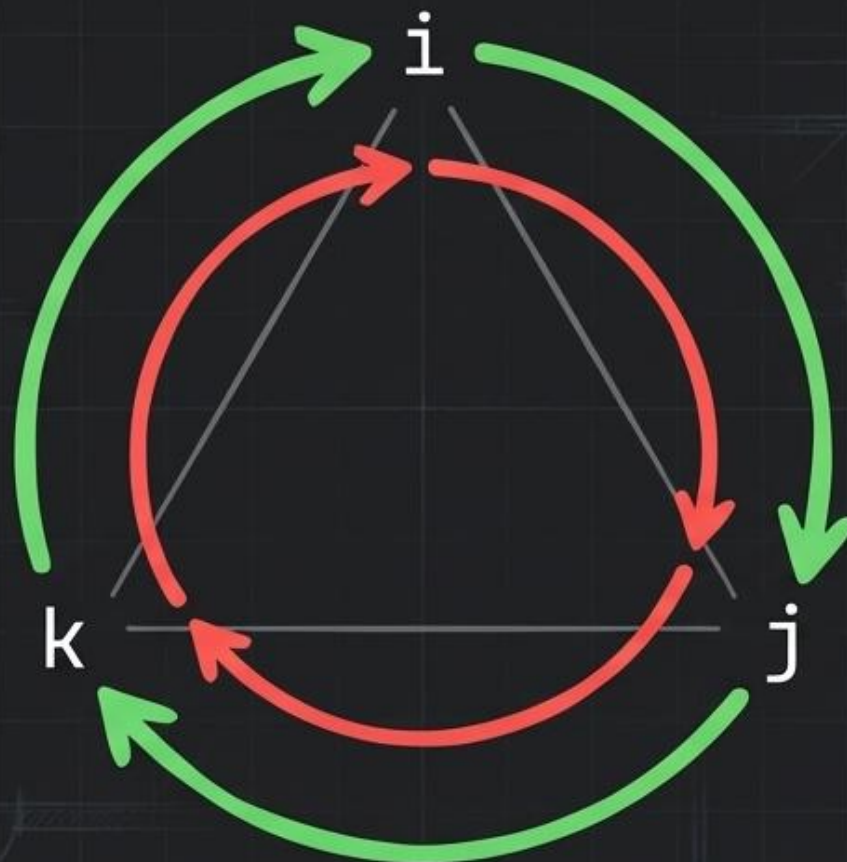
Rules of Imaginary Numbers

정방향 (Forward) $\rightarrow (+)$

$$ij = k$$

$$jk = i$$

$$ki = j$$



역방향 (Reverse) $\rightarrow (-)$

$$ji = -k$$

$$kj = -i$$

$$ik = -j$$

독특한 허수 곱셈 규칙 적용.

Unit Quaternion (q)

↓ 변환 (Conversion)

Rotation Matrix (회전 행렬)

$$\begin{bmatrix} a^2 + b^2 - c^2 - d^2 & 2bc - 2ad & 2bd + 2ac \\ 2bc + 2ad & a^2 - b^2 + c^2 - d^2 & 2cd - 2ab \\ 2bd - 2ac & 2cd + 2ab & a^2 - b^2 - c^2 + d^2 \end{bmatrix}$$

3D 그래픽스 파이프라인
적용을 위한
Matrix 변환 가능.

Orientation Interpolation (방향 보간)

Ideal Characteristics

1. Find minimum route
(최단 경로 탐색)
2. Uniform rotation speed
(일정한 회전 속도)



The Problem

**Euler
Representation
(오일러 방식)은
이 두 가지 조건을
만족하지 못함!**

Euler vs Quaternion

	Euler	Quaternion
Minimum Route	 불가	 가능
Uniform Speed	 불가	 가능

자연스럽고 정확한 3D 회전을 위해 Quaternion 필수.

SLERP (Spherical Linear Interpolation)

JetBrains Mono

Definition: 구면 선형 보간법.

Pretendard

Mechanism: 두 방향 ($q1$, $q2$) 사이를
쿼터니언으로 Interpolation.

Pretendard

Highlight Node

가장 자연스럽게 일정한 속도의 회전
애니메이션 생성.

Pretendard



Time (시간에 따른
부드러운 회전)

두 쿼터니언
사이의 각도

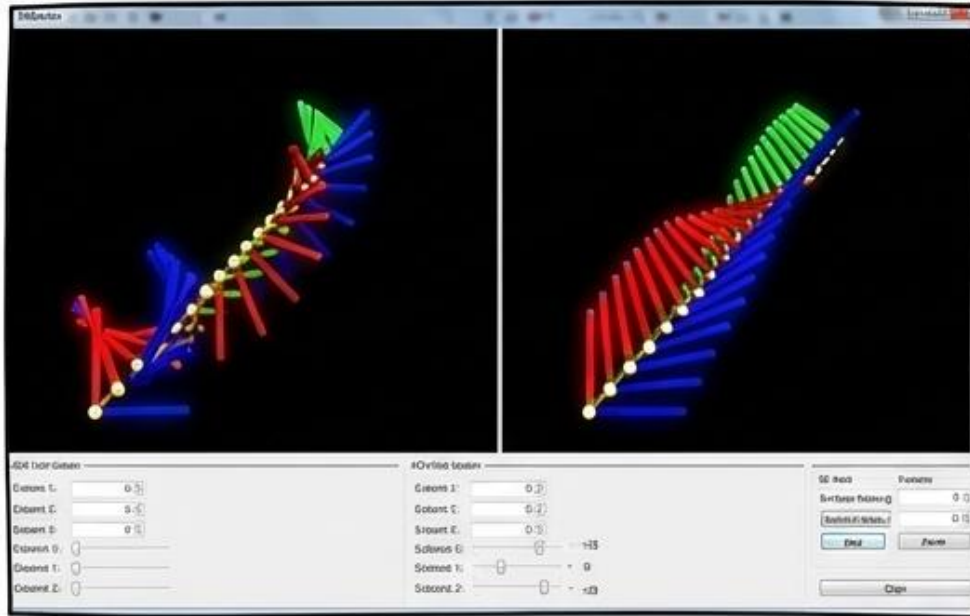
$$\mathbf{q}(t) = \frac{\sin \theta (1 - t)}{\sin \theta} \mathbf{q}_1 + \frac{\sin \theta t}{\sin \theta} \mathbf{q}_2$$

θ 계산을 위한 보조 공식 (JetBrains Mono)

$$\theta = \cos^{-1}(\mathbf{q}_1 \cdot \mathbf{q}_2)$$

$$\sin \theta = \sqrt{1 - (\mathbf{q}_1 \cdot \mathbf{q}_2)^2}$$

Where is it used?



3D Animation & Game Engines
(부드러운 관절/물체 회전)



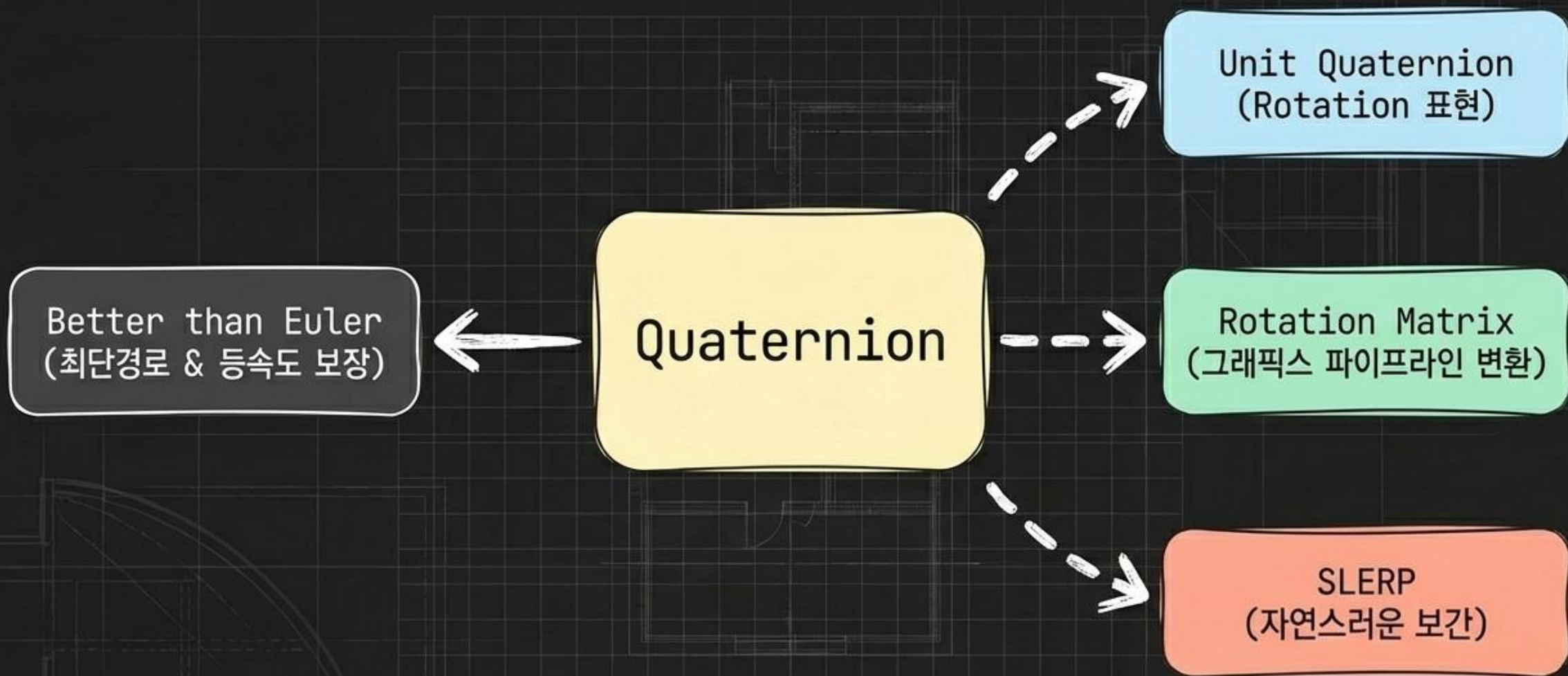
Drone Attitude Control
(드론의 안정적인 자세 제어)

Reference Note.

<https://www.youtube.com/watch?v=ltE0-evEOKI>

https://www.youtube.com/watch?v=0VAc_G79POE

Summary Map



Questions? 😊